SPIN BY OXXO · CASO DE NEGOCIO · FORWARD DEPLOYED ENGINEER

Operación Escaldada

Automatización end-to-end de contracargos y liquidación

Edgar Alberto Sanabria Paredes · 16 de octubre de 2026
Prototipo funcional: github.com/ragdeblacks/spin-conciliacion-contracargos
ragdeblack@gmail.com · linkedin.com/in/edgarsanp

Situación. Cada mañana un analista baja el CSV del portal del banco, lo cruza en Excel contra el Core de NetPay, libera liquidaciones por API y bloquea contracargos desde la interfaz del Core. El volumen creció 300 %.

Complicación. Mientras una persona esté en el camino crítico de cada transacción, más volumen significa más retraso, se contrate a quien se contrate. Además el banco cambia de portal en tres meses y libera una API en un año: cualquier solución pegada al portal de hoy nace con fecha de caducidad.

Respuesta. Propongo un pipeline diario, determinista y auditable, que bloquea contracargos antes de liberar una sola liquidación, mueve el dinero con idempotencia y rastro completo, deja a las personas solo en las excepciones y en los montos altos, y aísla detrás de adaptadores todo lo que va a cambiar. Entra a producción en tres fases con criterios de salida medibles. El prototipo ya corre de punta a punta y está en el repositorio.

Lo sostienen tres argumentos, que desarrollo en las páginas siguientes: **protege el dinero por diseño**, **sobrevive a los cambios del banco y del Core**, y **llega a producción sin sobresaltos**.

1. Protege el dinero por diseño

La automatización vale la pena solo si no puede perder dinero por su cuenta. Esa condición define cinco decisiones de diseño.

Los contracargos se bloquean primero y las liquidaciones se liberan después. Una liquidación tarde produce un comercio molesto; un contracargo sin bloquear produce una pérdida. El pipeline ejecuta todos los bloqueos del día y un gate verifica que no quede ninguno sin intentar antes de que arranque la primera liquidación. Las ventas de un comercio que ese día trae contracargo se retienen para que una persona decida si se liberan; esta regla es configurable y salió de una prueba en la que el Core simulado rechazó liberar dinero a un comercio recién bloqueado.

Ninguna acción se ejecuta dos veces. Cada liquidación y cada bloqueo llevan una clave única derivada del movimiento. La clave viaja al Core en la cabecera de idempotencia y se guarda en un libro mayor propio, de solo inserción. Repetir una corrida, reanudarla después de un corte o correrla contra un Core que devuelve estados atrasados no libera ni bloquea nada de nuevo. Hay una prueba automatizada para cada uno de esos tres escenarios.

El rastro se escribe antes de tocar el Core, y otra vez después. El libro mayor registra la intención, después la confirmación o la falla. Si el proceso muere a la mitad, se sabe qué quedó en el aire. Cada decisión guarda la regla que la produjo, la respuesta del Core, la evidencia (captura de pantalla cuando la acción fue por interfaz) y el nombre de quien aprobó.

Los montos altos y los patrones raros pasan por dos personas. Una liquidación arriba del umbral acordado con Operaciones, o un comercio con más de tres contracargos en el día, no se ejecutan solos: quedan pendientes hasta que un supervisor los aprueba con su nombre.

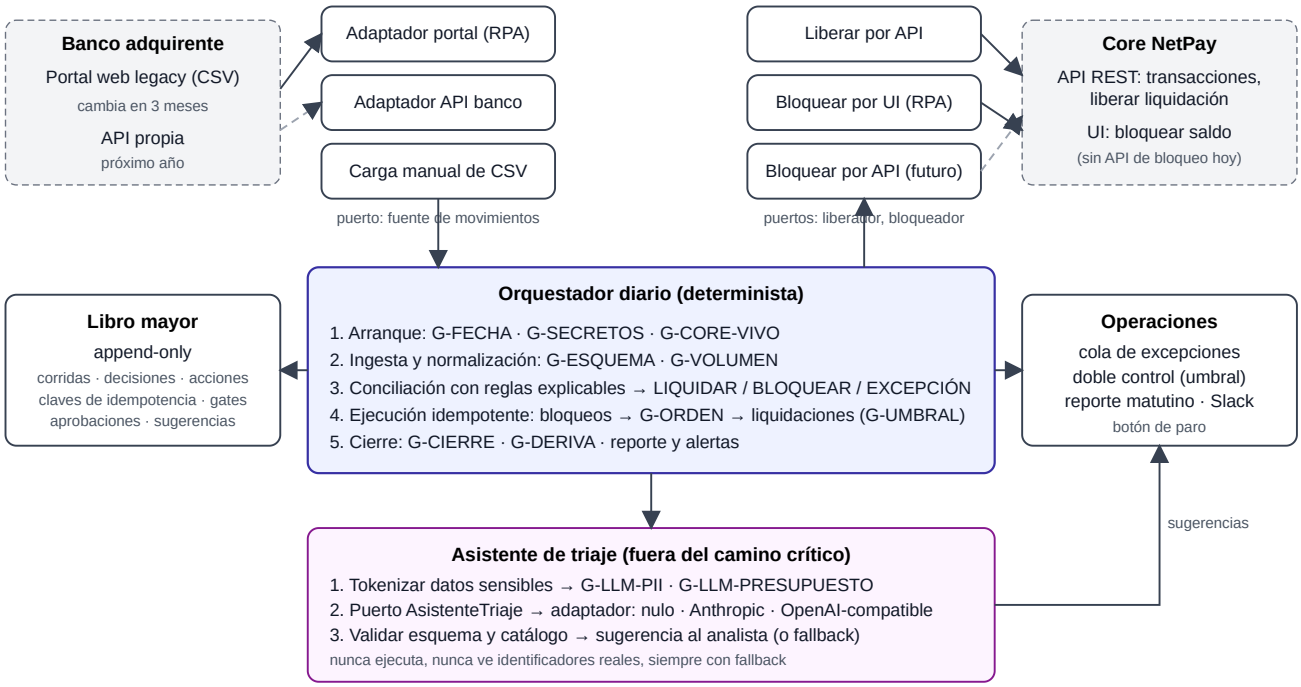
El sistema desconfía de su entrada. Un archivo vacío, uno con el triple de movimientos que el promedio histórico o uno con columnas distintas detiene la corrida antes de actuar. Al cierre se verifica que toda acción tenga estado final consistente; cualquier diferencia es una alerta. Los montos se manejan en centavos enteros, nunca en flotantes.

La inteligencia artificial no participa en nada de lo anterior. Entra únicamente en la cola de excepciones, para sugerirle al analista una causa probable y una acción de un catálogo cerrado, con los identificadores sustituidos por tokens antes de salir del proceso y con una respuesta determinista de respaldo si el modelo falla, no responde o se agota el presupuesto. La sección 4 detalla cómo.

Las credenciales viven en el gestor de secretos de la plataforma y llegan al contenedor como variables de entorno; el sistema se niega a arrancar si faltan o traen valores de relleno. El robot usa un usuario de servicio propio, separado de los analistas, para que la auditoría del Core distinga a la máquina de la persona. Los archivos y las capturas se cifran en reposo con la retención que acuerde Riesgos.

2. Sobrevive a los cambios del banco y del Core

El caso anuncia dos cambios y yo cuento un tercero que conviene provocar. Los tres se resuelven de la misma manera: lo que va a cambiar vive en un adaptador, fuera del núcleo, y se intercambia con una variable de configuración.



El núcleo solo conoce puertos. Cada sistema externo tiene su adaptador de hoy y, cuando aplica, el de mañana.

Cambio	Cuándo	Lo que se toca	Lo que no se toca
El banco cambia su portal	En 3 meses	Los selectores de pantalla, que son configuración, o el adaptador del portal. Una prueba de contrato diaria avisa el primer día que algo cambió. Mientras se ajusta, Operaciones sube el CSV a mano y el pipeline sigue	Reglas, libro mayor, ejecución, reporte
El banco libera su API	El próximo año	Se activa el adaptador de API, que el prototipo ya incluye contra una API simulada. Corre unas semanas en sombra junto al del portal, se comparan y se cambia la variable	Todo lo demás
El Core expone una API para bloquear	A negociar; es la primera petición que le haría al equipo del Core	Se activa el adaptador de API para bloqueo, que también existe ya, y se retira el robot sobre la interfaz	Todo lo demás
Cambia o se retira el modelo de IA	Cuando ocurra	La variable de modelo o de proveedor, después de pasar el set de evaluación	Todo lo demás

Sobre el robot que opera el portal y la interfaz del Core conviene ser claro: es la herramienta correcta mientras no haya API, y es frágil por naturaleza. Por eso cada acción deja captura, los selectores no están en el código y siempre existe el camino manual. La petición de la API de bloqueo al equipo del Core es la primera tarea del roadmap, porque elimina la mitad del robot.

3. Llega a producción sin sobresaltos

Un pipeline que corre en mi laptop no resuelve nada a las cinco de la mañana. Esta sección es la que, en mi experiencia, decide si una automatización se adopta o se abandona.

Tres modos, una variable. En modo *sombra* el pipeline decide y registra, pero no toca el Core; un comando compara después lo que habría hecho contra lo que hizo el analista. En modo *asistido* todo queda pendiente y el supervisor aprueba en lote. En modo *autónomo* ejecuta bajo umbrales y las personas solo ven excepciones y doble control. Cambiar de modo no requiere despliegue y queda registrado en cada corrida.

Fase	Semanas	Qué cambia	Para pasar a la siguiente
Discovery y MVP	1 a 2	Reglas extraídas del Excel y validadas con el analista; adaptador contra el portal real; despliegue en QA con un CSV real anonimizado	Operaciones aprueba las reglas
Sombra	3 a 5	Corre cada mañana en producción sin tocar dinero; comparación diaria contra el analista	Coincidencia mayor al 98 % durante tres semanas y cero falsos LIQUIDAR
Asistida	6 a 8	Ejecución real tras aprobación en lote; evidencia por acción; asistente de IA con proveedor real	Dos semanas sin fallas inexplicadas; aprobación en menos de 20 minutos
Autónoma	9 a 12	Ejecuta bajo umbrales	Las cuatro métricas de la tabla siguiente cumplidas dos semanas seguidas
Cambio de portal y API del banco	Mes 3 y cuando el banco la libere	Adaptador nuevo en sombra junto al viejo; cambio de variable	Coincidencia entre adaptadores mayor al 99.5 %

Métrica de éxito	Hoy	Objetivo
Hora en que queda liberada la liquidación del día	Cuando el analista termina	Antes de las 06:00
Movimientos conciliados sin intervención humana	0 %	Más del 95 %
Tiempo entre el reporte del contracargo y el bloqueo	Horas	Menos de 15 minutos desde la corrida
Liquidaciones duplicadas o a comercios con contracargo	Sin trazabilidad	Cero, con evidencia

Empaquetado y ejecución. Un contenedor construido sobre la imagen oficial de Playwright, configurado solo por variables de entorno e idéntico en desarrollo, QA y producción. Corre como tarea programada a las 05:00 (CronJob de Kubernetes o tarea programada del servicio de contenedores que use Spin), con reintento a las 05:20. Es un proceso por lotes con estado en base de datos, así que repetirlo o reanudarlo no duplica nada. El libro mayor va en Postgres; la evidencia, en almacenamiento de objetos cifrado.

Entrega continua. Cada cambio pasa por lint, pruebas unitarias, integración contra los servicios simulados, una prueba de robot real con navegador y la construcción de la imagen; el repositorio ya trae ese pipeline. La misma imagen se promueve a QA en automático y a producción con aprobación. Las reglas y los umbrales cambian por el mismo camino que el código, con revisión de Operaciones, nunca a mano en producción.

Observabilidad por fases. Desde la primera fase, logs estructurados con el identificador de corrida en cada línea, el resumen diario y las alertas en el canal de Slack de Operaciones, y un monitor de tipo *dead*

man's switch (Healthchecks o Cronitor) al que el pipeline le manda un latido al cerrar: si a las 05:30 no llegó, el monitor alerta por su cuenta, aunque el proceso haya muerto sin decir nada. Las alertas críticas también salen por correo, porque a esa hora el correo es lo que queda registrado y lo que Auditoría puede pedir. En la fase asistida se suma Sentry para capturar los errores no controlados con el contexto de la corrida y la versión desplegada, y Telegram para el supervisor de guardia, que a las cinco de la mañana mira el celular antes que Slack. En la fase autónoma, métricas por corrida (duración, acciones, excepciones, gates, consumo de tokens) en el tablero que Spin ya use, sea Datadog, Grafana o equivalente, con alertas por umbral y por deriva. Los canales ya están implementados en el prototipo detrás de un solo puerto de notificación; activar uno es definir su variable.

Si pasa esto	El sistema	Operaciones
El portal no responde o cambió	Detiene la corrida y alerta	Sube el CSV a mano y continúa; Tecnología ajusta los selectores
El Core no responde	Se detiene sin tocar nada y reintenta	Escala al equipo del Core; la corrida se repite después sin duplicar
Quedaron acciones fallidas al cierre	Las lista en el reporte	Las revisa, vuelve a correr o resuelve a mano y lo registra
El modelo de IA no responde	Usa la respuesta determinista	Nada
Hay que parar todo	Cambia a modo sombra o se suspende la tarea	Un comando del supervisor, sin despliegue

Riesgos que veo. Que las reglas del analista sean más complejas de lo que parecen; por eso el discovery es con el analista y la fase sombra dura tres semanas. Que el Core rechace acciones por reglas internas que nadie documentó; ya pasó en el prototipo y cada rechazo se vuelve una regla explícita. Que Operaciones no confíe en el sistema; la comparación visible en sombra y la última palabra humana en asistida existen para eso.

4. La evidencia

El flujo diario y sus controles

Paso (05:00)	Control	Si falla
Arranque: credenciales desde el gestor, Core vivo	G-SECRETOS, G-CORE-VIVO	No se toca nada; alerta
Ingesta del CSV por el adaptador activo	G-ESQUEMA, G-VOLUMEN	Se detiene; carga manual
Conciliación con reglas explicables	G-DUPLICADO	
Bloqueos por contracargo	G-UMBRAL	Acción fallida con rastro
Verificación de orden	G-ORDEN	No arrancan liquidaciones
Liquidaciones por API con idempotencia	G-UMBRAL	Acción fallida con rastro
Triaje de excepciones con el asistente	G-LLM-PII, G-LLM-PRESUPUESTO	El analista trabaja sin sugerencia

Paso (05:00)	Control	Si falla
Cierre, reporte y alertas	G-CIERRE, G-DERIVA, G-LATENCIA	Corrida marcada para revisión

Dónde entra la inteligencia artificial

Determinista, siempre	Modelo de lenguaje	Nunca un modelo
Ingesta, validación, normalización, conciliación, clasificación, ejecución, idempotencia, umbrales, cierre, auditoría	Triaje de excepciones: causa probable, acción sugerida de un catálogo cerrado, confianza y una explicación corta. En discovery, extraer las reglas del Excel del analista. Resumen matutino	Decidir liberar o bloquear. Llamar al Core. Cambiar reglas sin una persona

El modelo de referencia es Claude Sonnet 5.5; con cien excepciones diarias cuesta centavos al día. Va detrás de un puerto con tres adaptadores (Anthropic, cualquier servidor compatible con OpenAI, incluido un modelo abierto dentro del perímetro de Spin, y uno determinista), de modo que cambiar de proveedor es una variable. Los prompts y el contexto de negocio están versionados en el repositorio. Antes de llamar al modelo, el comercio y la referencia se sustituyen por tokens y un gate busca patrones de tarjeta, CLABE, CURP y correo. La respuesta se valida contra un esquema; si no cumple, entra la respuesta determinista. Un set de excepciones reales resueltas por el analista decide si un cambio de modelo o de prompt se despliega. Cada sugerencia queda registrada junto con lo que finalmente decidió la persona.

El prototipo

El repositorio contiene el portal del banco y el Core de NetPay simulados (incluida la API futura del banco, la API de bloqueo detrás de una bandera y un modo que imita el cambio de portal), el conciliador completo con los adaptadores descritos, datos de prueba con los casos incómodos (duplicados, montos distintos, contracargos sobre ventas ya liquidadas, acumulación en un mismo comercio, monto alto, cambio de esquema), 35 pruebas automatizadas, Docker Compose y el pipeline de integración continua.

Corrida real contra los servicios simulados	Resultado
Sombra, 0.9 s	186 movimientos: 143 a liquidar, 11 a bloquear, 30 excepciones con sugerencia, 2 ya procesados. No tocó el Core
Autónoma, 9.6 s	154 acciones: 10 bloqueos por la interfaz del Core con captura de evidencia y 141 liquidaciones por API. Dos decisiones quedaron en doble control, un supervisor las aprobó y se ejecutaron
Comparación sombra contra Core	154 de 154 coincidencias

Lo que el prototipo todavía no cubre: corre contra servicios simulados, el libro mayor es SQLite y las pruebas usan el adaptador determinista de IA para no depender de una llave ni gastar en integración continua. Los adaptadores reales existen y están validados contra el esquema.

```
make instalar && make mocks      # dependencias, navegador y servicios simulados
make demo                        # corrida en modo sombra
MODO=autonoma make demo          # corrida autónoma
make docker-demo                 # lo mismo, todo en contenedores
```